

AI Reallocation Model Development Plan

Here is a comprehensive, phase-by-phase development plan to build the **Category-Wise Capital Reallocation Predictive Model**, turning your dashboard into an automated inventory financial advisor.

Phase 1: Data Infrastructure & Feature Engineering (Step 1)

Goal: Upgrade the existing database and export pipeline so the Machine Learning (ML) model has enough categorized historical context to learn seasonal patterns.

- **Activities:**
 - Modify the database schema (or your `localStorage` /JSON structure) so every sales record is permanently tagged with the exact `sku` (Stock Keeping Unit), `product_id`, `category`, and `date`.
 - Write a script that takes the raw sales data and appends "Features" to it: `is_weekend`, `season` (based on date), and `is_holiday`.
 - Create an export function that spits out a clean CSV or JSON array containing at least 1-2 years of this enriched historical data.
 - **Outcome Achieved:** A fully enriched dataset where every transaction has SKU, category, and temporal context (e.g., *Record: SKU #CRD-500g (Curd) sold 85 units, on a Thursday, in Summer*).
 - **What to Achieve Next:** Now that the data is structured, you must build the "server" that can accept this data and run Python ML algorithms on it.
-

Phase 2: Python Forecasting Microservice (Step 2 & Step 5)

Goal: Create a bridge between your Node.js/Javascript frontend and the Python ecosystem (which is required for advanced models like XGBoost).

- **Activities:**
 - Set up a simple Python backend using **FastAPI** or **Flask**.
 - Create a POST endpoint (e.g., `http://localhost:5000/api/predict-reallocation`).
 - Write the skeleton code where this endpoint receives the JSON data from Phase 1, and eventually returns a mocked array response back to your Javascript app.
 - **Outcome Achieved:** A live, operational Python microservice that successfully talks to your main application via REST API.
 - **What to Achieve Next:** The microservice is currently hollow. It accepts data and returns mock data. In the next phase, you will replace the "mock data" by dropping the actual ML Model into this Python server.
-

Phase 3: Machine Learning Model Development (Step 3)

Goal: Train and deploy the XGBoost/LightGBM model to predict the next 30 days of sales *per product*.

- **Activities:**
 - Load the data inside the Python microservice using `pandas`.
 - Train an `XGBRegressor` model grouping the data by `category` and exact `sku`.
 - Force the model to output a 30-day forecast table containing `sku`, `product`, `forecasted_demand`, `current_stock`, and `cost_price`.
- **Outcome Achieved:** An intelligent forecasting engine that mathematically proves it understands that "Summer = High Dairy Demand, Low Pickle Demand." It outputs raw unit predictions per SKU.

- **What to Achieve Next:** Raw unit predictions (e.g., predicting 110 Curd sales) are not actionable yet. The next step takes these raw numbers and performs the financial math to create the "reallocation" advice.
-

Phase 4: Business Logic & Reallocation Engine (Step 4)

Goal: Build the "Virtual CFO" optimization algorithm that converts raw ML forecasts into rupee-based financial decisions.

- **Activities:**
 - Inside the Python microservice (or Node server), take the 30-day forecast numbers and run the math logic.
 - Calculate **Trapped Capital** (Surplus Stock × Cost Price).
 - Calculate **Required Capital** (Deficit Stock × Cost Price).
 - Cross-match products in the same category (or across the store) to generate the "Actionable Insight" pairings (e.g., "Shift ₹1,200 from Pickle to Curd").
 - Format the final output strictly into a standardized JSON payload.
 - **Outcome Achieved:** The backend pipeline is 100% complete. It now successfully ingests raw sales history, predicts the future, calculates the financial impact, and outputs human-readable Reallocation Rules.
 - **What to Achieve Next:** The backend is returning brilliant financial advice, but the user can't see it yet. The final phase involves displaying this gracefully on the front end.
-

Phase 5: UI/UX Implementation (Step 6)

Goal: Replace the legacy, useless aggregate charts in the dashboard with a clean, actionable "Intelligence Hub" Grid.

- **Activities:**
 - In `app.js`, intercept the JSON response from your Python endpoint.
 - In `index.html`, remove the old `<canvas id="predictionChart">` elements.
 - Build the dynamic **Resource Optimization Warning** action cards.
 - Use red (`#ef4444`) for "Reduce Inventory" text and green (`#10b981`) for "Increase Inventory" text.
 - *(Optional but powerful)* Add a "1-Click Apply" button on the cards that instantly modifies the user's upcoming Purchase Order list automatically.
- **Outcome Achieved:** A fully deployed, end-to-end Prescriptive Analytics system. The user opens the dashboard and is immediately greeted with smart, financial-saving inventory reallocation actions that prevent stockouts and eliminate dead stock.